

Final Project Report

RISC-V Cycle-Accurate Simulator

ENSC 254

July 29th, 2026

Lab Group:

Student Name	SFU ID
Dung Tien Nguyen	301472261
Jayden Lau-Lam	301611977
Jimmy (Yu Chao) Ma	301621233
Minhao Zhu	301541030

1. Milestone 1 — Basic Pipeline

1.1. Control Logic Generation (ID Stage)

Control signals are generated by `gen_control(instruction)` in `stage_helpers.h`. The function checks `instruction.opcode` and returns a partially filled `idex_reg_t` structure with `reg_write`, `mem_to_reg`, `mem_read`, `mem_write`, `branch`, `alu_src`, `alu_op`, and `is_jal` set according to the instruction type. `stage_decode()` then fills the remaining fields that `gen_control()` does not set, including the instruction bits, instruction address, `PC + 4`, the sign-extended immediate generated by `gen_imm()`, and the two source-register values

1.2. IF-Stage Mux Implementation

The mux at the PC input is implemented through three `pipeline_wires_t` fields. These are `pcsrc`, `pc_src0`, and `pc_src1`. `Pcsrc` is set during the previous cycle by `stage_mem` when a branch or jal is taken, and `pc_src1` holds the target. If `pcsrc` is false, fetch uses the sequential path. After fetching, `stage_fetch` updates the PC (`pc + 4`), then writes `regfile_p->PC = pwires_p->pc_src0` (unless a load-use stall is used)

1.3. EX-Stage Mux Implementation

This is implemented through the `ALUSrc` mux, selecting the ALU's second operand between `rs2_val` (forwarded) and the sign-extended immediate

`Alu_src` is set by `gen_control` per instruction type. The first ALU operand is always `idex_reg.rs1_val`, which by the time of execution will hold a forwarded value if needed

1.4. ALU Control Tables

alu_op values (set by `gen_control`, consumed by `gen_alu_control`):

alu_op	Meaning	Instructions
0	force ADD	loads, stores, lui, jal address math
1	force SUB	branches (compares rs1 vs rs2)
2	decode R-type funct3/funct7	add, sub, sll, slt, xor, srl, sra, or, and, mul, mulh, div, rem
3	decode I-type funct3	addi, slli, slti, xori, srli, srai, ori, andi

alu_control values (set by `gen_alu_control`, consumed by `execute_alu`):

Instruction	funct3	funct7	alu_control
add / addi	0x0	0x00	ALU_ADD
sub	0x0	0x20	ALU_SUB
mul	0x0	0x01	ALU_MUL
sll / slli	0x1	0x00	ALU_SLL
mulh	0x1	0x01	ALU_MULH
slt / slti	0x2	-	ALU_SLT
xor / xori	0x4	0x00	ALU_XOR
div	0x4	0x01	ALU_DIV
srl / srli	0x5	0x00	ALU_SRL
sra / srai	0x5	0x20	ALU_SRA
or / ori	0x6	0x00	ALU_OR
rem	0x6	0x01	ALU_REM

and / andi	0x7	-	ALU AND
------------	-----	---	---------

1.5. Branch Logic

Instruction	funct3	Condition
beq	0x0	zero
bne	0x1	!zero
blt	0x4	Signed, $rs1 < rs2$
bge	0x5	Signed, $rs1 \geq rs2$
bltu	0x6	Unsigned, $rs1 < rs2$
bgeu	0x7	Unsigned, $rs1 \geq rs2$

2. Milestone 2 — Hazard Detection & Forwarding

2.1. EX Hazard

An EX hazard is detected when the instruction in EX/MEM will write to a nonzero destination register, and that register matches a source register used by the instruction currently entering EX. In simplified form, the condition is: “EXMEM.reg_write && EXMEM.rd != 0 && EXMEM.rd == IDEX.rs1” or the equivalent condition for rs2. The logic also checks whether the current instruction actually uses that source register. When the condition is true, the forwarding mux selects the newer result from EX/MEM instead of the older value read from the register file.

2.2. MEM Hazard

MEM hazard is also checked in gen_forward, where it checks the instruction in the MEM/WB pipeline register. Again, like the EX hazard, it compares rd with rs1 and rs2. A MEM hazard is detected when MEM/WB will write to a nonzero destination register that matches rs1 or rs2 of the instruction entering EX. The MEM forwarding condition is only applied when an EX hazard does not already exist for the same operand. This gives the newer EX/MEM value priority in a double data hazard

2.3. Load-Use Hazard

A load-use hazard is detected in detect_hazard(). It checks whether the instruction entering the EX stage is a load, and whether the instruction currently being decoded needs the register being loaded. If so, the pipeline is stalled for one cycle. This is done by holding the PC, inserting a bubble into the ID/EX register, and freezing the IF/ID pipeline register

2.4. Control Hazard — Pipeline Flush

When a branch is taken, the processor sets pcsrc and provides the target address. In the meantime, the pipeline flushes the incorrect instructions that were fetched after the branch (due to PC+4 default guess) by replacing the IF/ID, ID/EX, and EX/MEM pipeline registers with bubbles

2.5. Stall / Bubble Insertion

Bubbles are utilized in both load-use and control hazards. These are NOP instructions with all control signals cleared so they don't affect the processor state

2.6. Other Pipeline-Stage Modifications

Several pipeline-stage changes were needed to support hazard handling. First, helper logic was added to determine whether each instruction actually reads rs1 or rs2, preventing false hazards for instruction fields that are not used as source registers. Second, EX forwarding was given priority over MEM forwarding because the EX/MEM value is newer during a double data hazard. Third, the forwarded operand values were carried into later pipeline registers so conditional branches could be evaluated using the correct values in the MEM stage. Finally, flushed instructions were replaced with 0x00000013, which represents addi x0, x0, 0, rather than all-zero instruction bits. This prevents the instruction decoder from treating a flushed pipeline slot as an invalid instruction

Sample from the current test run

Test	#Cycles	Fwd (EX-EX)	Fwd (EX-MEM)	Branches	Stalls
R	15	3	1	0	0
I	17	0	1	0	0
LS	22	1	1	0	0
random	27	6	4	1	0
multiply	145	2	17	16	0
vec xprod tiny	25354	4324	142	4136	60

3. Milestone 3 — Cache Integration

3.1. Manual Cycle Calculations (Test Category 1, Tests 1–4)

In this milestone, every load or store instruction accesses the cache during the MEM stage. Since the normal pipeline cycle is already counted, a cache hit adds 1 extra cycle (2 - 1), while a miss or eviction adds 101 extra cycles (102 - 1). We used these values with the number of hits and misses from each test. As shown below, the calculations match the #MEM stalls reported by the simulator

Test	Hits	Misses	Hits×1 + Misses×101	#MEM stalls	Match
LS	7	9	7 + 909 = 916	916	yes
multiply	1	1	1 + 101 = 102	102	yes
random	3	1	3 + 101 = 104	104	yes
testset 1	1	1	1 + 101 = 102	102	yes

To check the results further, we subtracted the memory stalls from the total cycle count. This gives the pipeline cycle count without the added memory-delay penalties. For the multiply and random tests, the results match their MS2 cycle counts exactly: 145 and 27 cycles. The LS test is 5 cycles higher because the MS3 input file includes five additional NOP instructions

Test	#Cycles	#MEM stalls	Base (#Cycles – stalls)	MS2 cycle count
------	---------	-------------	-------------------------	-----------------

LS	943	916	27	22 (MS3's LS.input has 5 extra NOP-padding instructions vs. MS2's)
multiply	247	102	145	145
random	131	104	27	27
testset 1	248	102	146	N/A (MS3-only test)

The table below summarizes the main statistics from the four Category 1 tests:

Test	#Cycles	Fwd(EX-EX)	Fwd(EX-MEM)	Branches	Cache acc.	Hits	Misses
LS	943	0	0	0	16	7	9
multiply	247	2	17	16	2	1	1
random	131	6	4	1	4	3	1
Testset 1	248	2	3	16	2	1	1

3.2. Cache Performance Impact

The cache clearly improves the program's performance. Without a cache, every load or store pays the full 100-cycle memory latency. With the single-level cache enabled, 236,642 out of 249,088 accesses hit in the cache, giving a 95.0% hit rate. This reduces the total cycle count from 26,935,748 to 3,769,724. This result makes sense because `vec_xprod.input` uses tiled loops and repeatedly accesses nearby data, so the program has both temporal and spatial locality

3.3. Speedup Analysis

For `vec_xprod.input`, the speedup is $26,935,748 / 3,769,724 = 7.15x$. Therefore, the single-level cache makes the program about 7.15 times faster than the no-cache version

3.4. Cache Configuration Exploration (DSE)

To find the best configuration for each cache size, we tested different combinations of set count, associativity, block size, and replacement policy using `vec_xprod.input`. For each cache size, we selected the configuration that produced the highest hit rate. The selected settings are stored in `dse_1k.txt`, `dse_2k.txt`, `dse_4k.txt`, and `dse_8k.txt`

File	-s	-E	Policy	-b	Size check
dse_1k.txt	0	8	LRU	7	$2^0 \times 8 \times 2^7 = 1024 \text{ B}$
dse_2k.txt	0	8	LRU	8	$2^0 \times 8 \times 2^8 = 2048 \text{ B}$
dse_4k.txt	0	8	LRU	9	$2^0 \times 8 \times 2^9 = 4096 \text{ B}$
dse_8k.txt	0	8	LRU	10	$2^0 \times 8 \times 2^{10} = 8192 \text{ B}$

The next table summarizes the cycle counts, hit rates, and speedups obtained from our DSE runs. The no-cache result of 26,935,748 cycles was used as the baseline for calculating speedup

Cache size	#Cycles	Hits	Misses	Hit rate	Speedup
1 KiB	2,666,524	247,674	1,414	99.432%	10.10×
2 KiB	2,595,624	248,383	705	99.717%	10.38×
4 KiB	2,560,224	248,737	351	99.859%	10.52×
8 KiB	2,541,024	248,929	159	99.936%	10.60×

All four selected configurations use one fully associative set ($s = 0$), 8 cache lines, and LRU replacement. As the cache size increases, the block size also increases. This works well for `vec_xprod` because the tiled loop keeps reusing a small group of addresses and also reads nearby data in sequence. Full associativity reduces conflict misses, while the larger blocks make better use of spatial locality. The hit rate increases from 99.432% for the 1 KiB cache to 99.936% for the 8 KiB cache, and the speedup increases from 10.10x to 10.60x

4. Milestone 4 — Innovations

4.1. Feature: Two-Level Cache (L1-I/D + Unified L2)

L1 is split into a dedicated instruction cache (L1-I, checked in `stage_fetch`) and data cache (L1-D, checked in `stage_mem`), each backed by a larger unified L2 that both access after an L1 miss. The existing framework creates and passes the L1-D cache object into `cycle_pipeline()`, while L1-I and the unified L2 are static cache objects managed in `pipeline.c`. The cache lookup, replacement, and setup helpers remain in `cache.c`, allowing `riscv.c` to remain byte-for-byte identical to the framework copy

Test cases:

- Functionality (`two_level_functionality`): `LS.input` run with full per-cycle and cache tracing, diffed against a golden reference
- Performance (`two_level_performance`): `vec_xprod.input` was run in stats-only mode with the two-level cache enabled and both `BRANCH_PREDICT_1BIT` and `BRANCH_PREDICT_2BIT` undefined. This gives a clean cache-only comparison against the MS3 single-level baseline
- Corner case (`two_level_corner`): `random.input`'s short, scattered access pattern evaluates cold-start behavior and limited reuse across L1-I, L1-D, and L2

Commands:

```
bash test_simulator_ms4.sh two_level_functionality
```

```
bash test_simulator_ms4.sh two_level_performance
```

```
bash test_simulator_ms4.sh two_level_corner
```

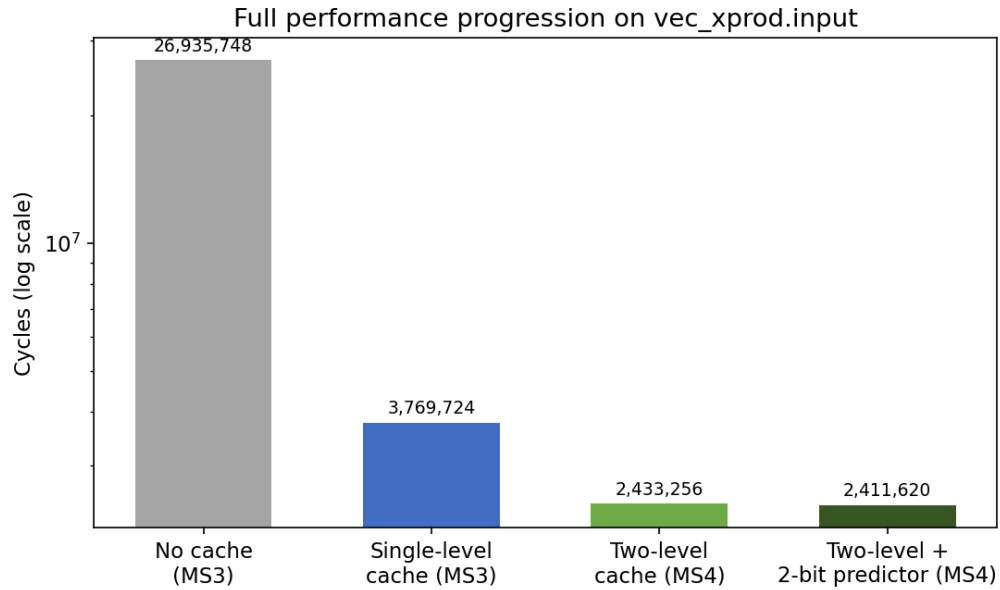


Figure 1. Full performance progression on vec_xprod.input: no cache, MS3 single-level, MS4 two-level, and MS4 two-level + 2-bit predictor combined

With both `BRANCH_PREDICT_1BIT` and `BRANCH_PREDICT_2BIT` undefined, the two-level cache provides a $1.55\times$ speedup over the MS3 single-level cache (3,769,724 to 2,433,256 cycles) and an $11.07\times$ speedup over the no-cache baseline (26,935,748 to 2,433,256 cycles). In this implementation, the reported MS4 memory-stall counter records latency beyond the normal L1 lookup: an L1 hit adds 0 extra cycles, an L2 hit adds 10 cycles, and an L2 miss adds 110 cycles. Therefore, $12,125(10) + 327(110) = 157,220$ extra cycles, exactly matching the reported `#MEM stalls` value. The L1-D split of 236,642 hits and 12,446 misses is identical to the MS3 single-level result, so the comparison isolates the benefit of adding L2. The unified L2 receives 12,452 combined misses from L1-I and L1-D. It hits on 12,125 of these accesses, leaving only 327 accesses to reach main memory

Corner case data (random.input, two level corner):

Metric	Value
#Cycles	357
#Branches taken	1
L1-I accesses / hits / misses	27 / 25 / 2
L1-D accesses / hits / misses	4 / 3 / 1
L2 accesses / hits / misses	3 / 0 / 3

On this short, single-branch program, all 3 L2 accesses miss (0 L2 hits). The program finishes before the L2 cache sees useful reuse, so the L2 provides no additional hit benefit in this test case. This is a useful cold-start corner case rather than a failure of the hierarchy

4.2. Feature: 1-Bit and 2-Bit Branch Predictor

Both predictors use a 64-entry Branch History Table (BHT) indexed by $(PC \gg 2) \& 63$, corresponding to PC bits [7:2]. The 1-bit predictor stores the last observed branch outcome and initializes every entry to not taken. The 2-bit predictor initializes every entry to weakly not taken and predicts taken when the saturating-counter state is 2 or 3. A 2-bit entry must receive two consecutive contradictory outcomes to move from a strong prediction to the opposite direction, which makes it more resistant to isolated loop exits. Prediction occurs in the IF stage; the actual outcome from `gen_branch()` is resolved in MEM, updates the selected BHT entry, and invokes the existing MS2 flush mechanism only when the prediction is incorrect

Test cases:

- Functionality (`bp1_functionality` / `bp2_functionality`): `multiply.input` run with full tracing, diffed against golden references for both predictor variants
- Performance (`bp1_performance` / `bp2_performance`): `vec_xprod.input` run stats-only, isolating each predictor's own contribution against the two-level-cache-alone baseline
- Corner case (`bp1_corner` / `bp2_corner`): `random.input`'s single branch, exercising cold-start behaviour

Commands:

```
bash test_simulator_ms4.sh bp1_functionality # and bp2_functionality
```

```
bash test_simulator_ms4.sh bp1_performance # and bp2_performance
```

```
bash test_simulator_ms4.sh bp1_corner      # and bp2_corner
```

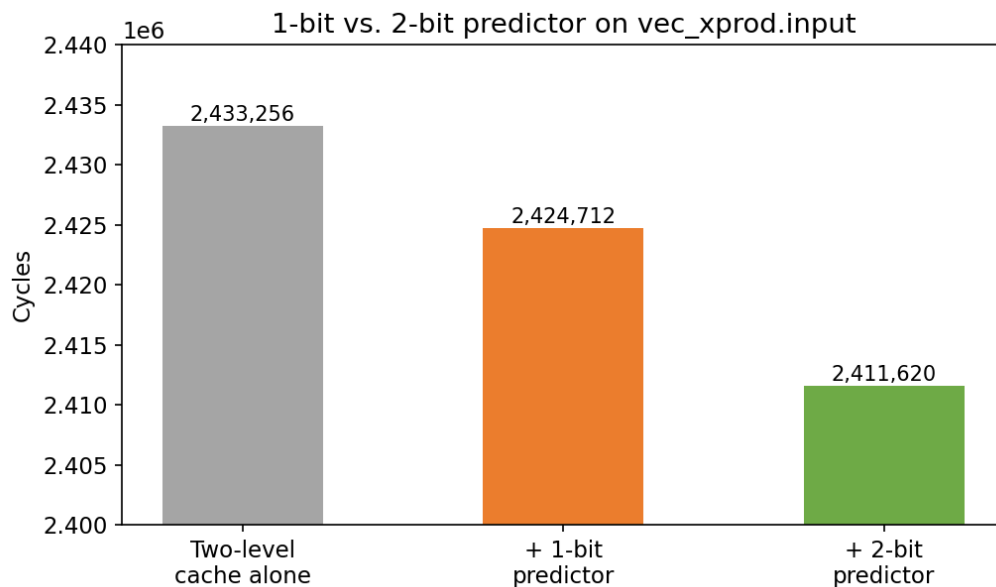


Figure 2. 1-bit vs. 2-bit predictor, both measured against the two-level-cache-alone baseline on `vec_xprod.input`

Effectiveness vs. expectations:

Configuration	#Cycles	Accuracy	Cycles saved vs. cache-alone
Two-level cache alone	2,433,256	—	—

+ 1-bit predictor	2,424,712	93.91%	8,544
+ 2-bit predictor	2,411,620	96.95%	21,636

The 2-bit predictor saves approximately $2.5\times$ as many cycles as the 1-bit predictor in this test case. This result is consistent with the expected behavior of a 2-bit saturating predictor: it resists changing direction after a single atypical outcome, such as the not-taken exit of a branch that was taken throughout a loop, whereas the 1-bit predictor immediately records the latest outcome

The corner case is the most instructive result: on `random.input` (a single branch), both `bp1` and `bp2` record 1 lookup, 0 correct, 1 mispredict, and identical total cycle counts (357, matching the no-predictor baseline exactly). This is expected, not a defect — with no history at all, the very first branch either predictor ever sees is guaranteed to be a cold-start misprediction if it happens to be taken, and with only one branch total, both predictors behave identically since there is no opportunity for their designs to diverge

Configuration (<code>random.input</code>)	#Cycles	BP lookups	BP correct	BP mispredicts	Accuracy
Two-level cache alone	357	—	—	—	—
+ 1-bit predictor	357	1	0	1	0.00%
+ 2-bit predictor	357	1	0	1	0.00%

5. Conclusion

- MS1: all 5 tests pass (R, I, LS, random, multiply)
- MS2: all 6 tests pass (R, I, LS, random, multiply, `vec_xprod_tiny`)
- MS3: all 6 tests pass (4 test-category-1 tests, cache summary, no-cache baseline); cache gives a $7.15\times$ speedup on `vec_xprod.input`
- MS4 two-level cache: $1.55\times$ speedup vs. MS3's single-level cache (isolated, no predictor), and $11.07\times$ vs. no cache at all
- MS4 predictors: the 2-bit predictor reaches 96.95% accuracy (compared with 93.91% for the 1-bit predictor) and saves approximately $2.5\times$ as many cycles on `vec_xprod.input`, consistent with the expected resistance of a 2-bit saturating counter to isolated branch outcomes